Software Requirements Specification

for

Byte Your Fork

Version 1.0 approved

Prepared by Sous Chef

2-16-2026

Table of Contents

1. Introduction 3
1.1 Purpose 3
1.2 Intended Audience and Reading Suggestions 3
1.3 Scope 3
Overall Description 3
2.1 Product Perspective 4
2.2 Product Functions 4
2.3 User Classes and Characteristics 5
2.3.1 Home cook / Normal User Characteristics 6
2.3.1a User Inputs 6
2.3.2 Admin/Recipe Manager: 6
2.3.2a Admin Inputs 6
2.4 Operating Environment 7
2.5 Design and Implementation Constraints 7
2.6 Assumptions and Dependencies 8
2.6.1 Assumptions 8
2.6.2 Dependencies 8
3. External Interface Requirements 9
3.1 User Interfaces 9
3.2 Hardware Interfaces 9
3.3 Software Interfaces 10
3.4 Communications Interfaces 10
4. System Features 11
4.1 User Account and Profile Management 11
4.1.1 Description and Priority 11
4.1.2 Stimulus/Response Sequences 12
4.1.3 Functional Requirements 12
4.2 Recipe Discovery and Visualization 12
4.2.1 Description and Priority 12
4.2.2 Stimulus/Response Sequences 12
4.2.3 Functional Requirements 12

4.3 Interactive Voice-Guided Cooking 13
4.3.1 Description and Priority 13
4.3.2 Stimulus/Response Sequences 13
4.3.3 Functional Requirements 13
5. Other Nonfunctional Requirements 13
5.1 Performance Requirements 13
5.2 Safety Requirements 13
5.2.1 User privileges 14
5.2.2 Encryption 14
5.2.3 Tokens 14
5.3 Security Requirements 14
5.3.1 Password-Hashing 14
5.3.2 SQL injection prevention 14
5.3.3 Authentication 14
5.3.4 Voice Data Privacy 14
5.3.5 Security Logging 14
5.3.6 Authorization and Access Control 14
5.4 Software Quality Attributes 15
5.4.1 User Quality of Life 15
Appendix A: Glossary 15
Appendix B: Analysis Models 18
System Context Diagram 19
Data Flow Diagram 19
ERD Diagram 20
Sample Login Page 22

1. Introduction

This Software Requirements Specification is an overview of the agreement between the customer and the developers of Byte Your Fork (BYF) application. This is a statement of the functionality, external interfaces, attributes, and design constraints of the application.

1.1 Purpose

The purpose of this document is to specify the functional and non-functional requirements of BYF. BYF will allow for recipe browsing, hands-free voice-guided cooking sessions, and timer coordination. This specification does not cover the how but the what of this software.

1.2 Intended Audience and Reading Suggestions

The intended audience of this document includes stakeholders (e.g., customers, management) and any individuals requiring detailed technical documentation of BYF.

1.3 Scope

Byte Your Fork is a user-facing, interactive cooking application. The system supports recipe search and browsing, voice-controlled step-by-step guidance, ingredient tracking, and cooking session management. The application includes both visual interface and voice-tracking capabilities.

The system is designed to provide automatic support for the following processes:

1. Discovering and browsing recipes.
2. Following step by step cooking instructions via voice narration.
3. Managing multiple simultaneous cooking timers.
4. Tracking cooking session(s) progress.

The system is a web-based application accessible via modern web browsers on desktop computers, tablets, and mobile devices. The system will be hosted on cloud infrastructure with users accessing it through a standard web interface.

Overall Description

This section includes details about what is and what is not expected of the Byte Your Fork system, intentionally unsupported cases, and assumptions used in system design.

2.1 Product Perspective

Byte for Fork is a new, self-contained web application developed by Sous Chef to address the growing demand for hands-free cooking assistance in the modern kitchen. Unlike traditional methods, such as trying to flip through a cookbook with busy hands, Byte Your Fork introduces voice control to streamline a user's cooking experience.

This is not a replacement for any existing system, nor is it a component to a larger system but its own standalone system. The application emerged from research indicating that home cooks frequently:

- Struggle to read recipe instructions while actively cooking.
- Lose track of timers when preparing complex meals.
- Give up a recipe due to confusion about ingredients or steps.
- Find cookbooks cumbersome to deal with in practical cooking.

2.2 Product Functions

Byte Your Fork will provide the following core functions; below are high-level descriptions to which you can see the context model diagram for in appendix B:

Recipe Management

- Maintain a database of recipes with ingredients, steps, and metadata.
- Include a title, description, cuisine type, and cooking time for each recipe.
- Support recipe categorization by cuisine and dietary restrictions.
    - Allow recipe search by name, cuisine, or dietary tags.
- Display recipes with images, ratings, and user reviews.

User Account Management

- Allow users ("chefs") to create accounts with email and password.
    - Federated Sign in with an Identity Provider will also be supported.
- Include a unique username, hashed password, email, and dietary preferences for each user profile.
    - Users will be able to change their email and passwords but not their username.
- Support user login and log out functionality.
- Auto-logout after 30 minutes of no activity.
- Admins ("head chefs") will be able to moderate problematic users.

Recipe Discovery and Display

- Showcase a browsable list of available recipes.
- Allow filtering by cuisine type, dietary restrictions, difficulty, and maximum time to prep.
- Provide a detailed recipe with a complete ingredient list and instructions.

Voice-Guided Cooking Sessions

- Allow users to start a hands-free cooking session for any recipe.
- Read the ingredient list aloud on request.
- Narrate each request with text-to-speech software.
- Support for voice commands, e.g., “next step”, “set a timer”, “previous step”, “pause” and “resume”.
- Provide visual confirmation of recognized voice commands.
- Maintain a session if the app is closed or reopened.

Timer-Management (Multi-Threaded)

- Automatically start timers for steps with duration i.e. “Bake for 30 minutes”.
- Support multiple simultaneous timers up to five timers.
- Each timer runs on a dedicated thread for independent operation.
- Display visual countdowns for all times.
- Provide visual and audio alerts when timers are completed.
- Support timers pause, reset, and resume via voice control.

User Favorites and History

- Allow users to save recipes to favorites (“forks”)
- Track cooking session history, e.g., date, recipe, and completion status.
- Prompt users to rate recipes after completing a cooking session.
- Display a user’s past cooking sessions.

Admin Recipe Management

- Allow admins to create new recipes with full details
- Enable editing of existing recipes, e.g., title, ingredients, steps, metadata.
- Support recipe image upload.
- Allow admins to categorize recipes with tags.
- Provide ability to delete recipes.

2.3 User Classes and Characteristics

2.3.1 *Home cook / Normal User Characteristics*

The typical Byte Your Fork user is a home cook who:

- has basic computer literacy and can navigate a web browser.
- owns a device with speaker and microphone capabilities.
- has Internet access.

- ranges from beginner to expert cooking skill.
- may have dietary restrictions and/or cuisine preferences.
- seeks a hands-free cooking assistant solution.

2.3.1a User Inputs

- Personal Information: Full name, email, password.
- Dietary Preferences: Vegetarian, vegan, gluten-free, dairy-free, Kosher.
- Skill-Level: Beginner, intermediate, expert.
- Cuisine Preferences: Mexican, Italian, French, Asian, Mediterranean.
- Recipe Reviews: Users may comment on recipes and leave a review up to five stars.

2.3.2 Admin/Recipe Manager:

The Head Chef is a recipe curator or content manager who:

- has above-average technical literacy.
- understands recipe structure and cooking jargon.
- can write clear and concise cooking instructions.
- has permission to create, edit, and delete recipes.
- owns responsibility for maintaining recipe quality and accuracy.

2.3.2a Admin Inputs

- Recipe Creation: Recipe name, description, cuisine type, cooking time, difficulty level**.**
- Ingredients: Ingredient name, ingredient quantity, preparation notes.
- Instruction: Sequential steps of recipe completion.
- Recipe Image: JPEG, HEIC.
- Categorization: Allow admins to attach categories to recipes.
- Review Management: Admins shall be able to flag inappropriate content and delete spam comments.
- User Moderation: Admins will be able to warn and ban users and reset passwords.

The system assumes Head Chefs:

- will not use a second user account.
- have been trained on formal recipe entry practices.
- understand dietary categorization and tagging.

2.4 Operating Environment

BYF is a web application accessible by any web browser on all standard operating systems (Windows, Mac, Linux). The infrastructure for the web application will be hosted by a cloud service provider owned by the Texas Cyber Range.

Server Environment

- Web Server: Node.js (v16+).
- Database: PostgreSQL (v13+).
- Operating System: Ubuntu 24 Server LTS.
- Hosting: Cloud Instance provisioned by Texas Cyber Range.
- HTTPS: Required for all client communication.

Backend Architecture

- Multi-threaded timer service.
- User authentication and authorization service.
- Session State Management.
- Recipe search engineer.

External Service Dependencies

- Speech Recognition: Gemini API (formerly Google Voice API).
- Email Service: Node Mailer (2FA).
- Identity Provider: Used for federated sign on.

2.5 Design and Implementation Constraints

Voice Control Limitations

- Heavy background noise from a busy kitchen can hinder the output of the program.
- Microphone quality affects voice recognition.

Browser and Device Requirements

- The application must be used on modern browsers such as Google Chrome, Safari, Firefox and Microsoft Edge.
- Devices must have JavaScript enabled.
- Devices must have microphone and speaker permissions enabled.

Platform Restraints

- Byte-your-fork is web based; native mobile apps are not included.

Security Scope

- Before final deployment, the Byte Your Fork will undergo a white-box application penetration test.

Content Limitations

- Recipe database must be pre-seeded although web-scraping sites for recipes will be utilized.
- Video tutorials are not included.
- Kitchen equipment, shopping lists, and nutritional calculation are not in scope of this application.

2.6 Assumptions and Dependencies

2.6.1 Assumptions

- **Constant Connectivity:** The system assumes users have a stable, high-speed Internet connection during cooking sessions. Since audio streaming via WebSockets is central to the "Cook Mode," significant packet loss or latency will degrade the voice assistant's responsiveness.
- **Environment Suitability:** It is assumed that the user's kitchen environment provides a signal-to-noise ratio sufficient for the Gemini Multimodal Live API to distinguish human speech from ambient kitchen sounds (e.g., running water, sizzling pans, or vent fans).
- **Browser Capabilities:** The SRS assumes that the user is utilizing a modern web browser that fully supports the Web Audio API and MediaDevices API for microphone access and real-time audio processing.
- **Administrative Integrity:** The system assumes that individuals granted "Head Chef" (Admin) status have been vetted and will provide accurate, safe, and high-quality recipe data.

2.6.2 Dependencies

- **Google Gemini Multimodal Live API:** The core functionality of the hands-free voice assistant is strictly dependent on the availability and up time of the Gemini API. Any service outages or breaking changes to this third-party API will directly affect the "Cook Mode" feature.
- **Cloud Infrastructure:** The application depends on cloud service providers (e.g., AWS, Azure, or Google Cloud) for hosting the web server, managing the RDBMS, and maintaining the WebSocket gateway.
- **SMTP Service Provider:** Asynchronous communications, such as password resets and account verifications, are dependent on a third-party transactional email service (Nodemailer).
- **Identity Providers:** Federated sign-in capabilities depend on the continuous availability and integration support of external identity providers using Google.

3. External Interface Requirements

3.1 User Interfaces

The user interacts with the application through a responsive web interface built using the React framework and served via Vite. The interface is designed to be accessible on both desktop and mobile viewports via the browser. The following specific screens define the user experience:

- Authentication Screen: Provides input fields for email and password login, alongside "Sign Up" and "Forgot Password" workflows. It must include clear error states for invalid credentials.
- Recipe Discovery Dashboard: Features a prominent search bar and filter chips (e.g., "Vegan", "Italian") at the top. The main view renders a grid of recipe cards displaying the image and when hovered over the title and star rating appear.
- Cook Mode Interface: A specialized view optimized for distance reading. It features a visual audio waveform indicator that animates when the application is listening to the user's voice or speaking back, providing necessary feedback that the Gemini agent is active.

3.2 Hardware Interfaces

- Client Computing Device: The application requires a device capable of running a modern web browser (e.g., Chrome, Firefox, Safari).
- Audio Input (Microphone): A functional microphone is strictly required for the voice assistant feature. The software interfaces with this hardware via the browser's Media Devices API to capture audio at a minimum sample rate of 16kHz.
- Audio Output (Speakers): The device must include a speaker to render the Text-to-Speech (TTS) audio response generated by the Gemini API.
- Network Interface: A broadband internet connection is required to sustain the low-latency WebSocket connection needed for real-time voice streaming.

3.3 Software Interfaces

The backend exposes the following API which client devices communicate with via the frontend:

- /api/signup
    - The user creates a new account and provides their account information

- /api/login
    - The user provides their username and password in exchange for a session token
- /api/search
    - A search phrase from the user is used to query the database for relevant recipes
- /api/recipeInfo
    - Specific information about a recipe such as cooking instructions are returned
- /api/startRecipe
    - The user initiates the cooking process by providing the recipe they want to cook. The frontend then opens a WebSocket connection to the backend for bi-directional communication during the cooking process.

External Voice Processing Interface

The system shall interface with the external Google API to provide natural language understanding and speech synthesis capabilities. This interface must support a real-time, bidirectional stream of audio data, enabling the system to capture user speech and play back synthesized voice responses with minimal latency.

The primary function of this interface is to offload the computational complexity of speech recognition and intent classification, accepting raw audio as input and returning structured commands and synthesized speech as output. The system must mediate this connection through its own backend infrastructure to enforce security policies and manage session state.

Persistent Data Storage Interface

The backend system interfaces with a Relational Database Management System (RDBMS) to ensure the durability and integrity of application data. This interface is responsible for the structured storage and retrieval of static content, such as recipe instructions and metadata, as well as dynamic transactional data, including user profiles and active session progress. The interface must support efficient querying mechanisms to retrieve specific data subsets based on user context, serving as the definitive record for the application state across all user interactions.

3.4 Communications Interfaces

Client-Server Communication Protocols

The system requires a web-based communication architecture utilizing standard Hypertext Transfer Protocol Secure (HTTPS) for all transactional data exchange. This ensures that sensitive user data, including authentication of credentials and personal profile information, is encrypted in transit using Transport Layer Security (TLS).

The communication model must support a request-response pattern for standard operations, such as content browsing and account management, where the client initiates requests, and the server provides structured data payloads.

Real-Time Voice Streaming Synchronization

To support the voice assistant functionality, the system must implement a persistent, full-duplex communication protocol. This interface is required to maintain an open channel for continuous audio streaming between the client and the server, eliminating the latency overhead associated with repeated HTTP handshakes. The system must enforce strict synchronization between the audio stream and the visual interface, ensuring that state changes triggered by voice commands are immediately reflected in the client's application's view.

Message Formatting and Serialization

All structured data exchanged across these interfaces needs to be serialized in a lightweight, human-readable format (such as JSON) to ensure interoperability between the client interface, the backend server, and external integration points. Binary data, specifically audio streams, must be transmitted using standard Pulse Code Modulation (PCM) or similar uncompressed formats to maximize compatibility with the external voice processing service.

Electronic Mail Communication Protocols

All asynchronous user notifications and system alerts shall be transmitted using standard email transmission protocols (such as SMTP) over secure, encrypted channels (STARTTLS or SSL). The system must format these communications as multi-part MIME messages, supporting both plain text and HyperText Markup Language (HTML) representations to ensure compatibility across a wide range of email clients. This communication channel is strictly reserved for outbound transactional messages required for the maintenance of user accounts and the delivery of requested content

4. System Features

4.1 User Account and Profile Management

4.1.1 Description and Priority

This feature allows users to establish a persistent identity within the application. It is a high-priority feature, as it is a prerequisite for saving personal preferences, dietary restrictions, and creating custom shopping lists. The system must support both standard email registration and federated identity verification.

4.1.2 Stimulus/Response Sequences

The user navigates to the registration page and inputs their credentials. The system validates the input format and uniqueness. Upon successful validation, the system creates a new database record and grants the user access to authorized features. If the user attempts to register with an existing email, the system displays a validation message requiring a unique address.

4.1.3 Functional Requirements

- REQ-1: The system shall accept user registration via email and password, ensuring passwords are hashed before storage.
- REQ-2: The system shall allow users to configure dietary preferences (e.g., vegan, gluten-free) which serve as filters for recipe discovery.
- REQ-3: The system shall enforce an automatic session timeout after 30 minutes of inactivity to ensure account security.
- REQ-4: The system shall provide a mechanism for users to reset their authentication credentials via an asynchronous email verification process.

4.2 Recipe Discovery and Visualization

4.2.1 Description and Priority

This feature constitutes the primary interface for content consumption. It is a high-priority feature that enables users to browse, search, and view recipe details. The system must index recipes by title, ingredients, and metadata tags to facilitate efficient retrieval.

4.2.2 Stimulus/Response Sequences

The user enters a search term or selects a category filter. The system queries the persistent storage and returns a paginated list of matching results. When a user selects a specific item, the system retrieves and renders the full recipe details, including the ingredients list, step-by-step instructions, and associated media.

4.2.3 Functional Requirements

- REQ-5: The system shall provide a search interface capable of querying recipes by title, ingredient name, or cuisine type.
- REQ-6: The system shall filter search results based on the user's stored dietary preferences by default, while allowing manual overrides.
- REQ-7: The system shall display recipe details in a responsive layout, separating the ingredient list from the instructional steps for readability.
- REQ-8: The system shall allow authenticated users to save specific recipes to a "Favorites" list for rapid future access.

4.3 Interactive Voice-Guided Cooking

4.3.1 Description and Priority

This is the core differentiator of the application and is classified as a high-priority feature. It enables hands-free interaction with the recipe instructions. The feature utilizes the external voice processing interface to interpret spoken commands and provide auditory feedback, allowing the user to cook without physical interaction with the device.

4.3.2 Stimulus/Response Sequences

The user activates "Cook Mode" on a specific recipe. The system initializes the bidirectional audio stream. The user issues a verbal command such as "next step." The system processes the audio, identifies the intent, advances the recipe cursor in the database, synthesizes the instruction text into speech, and plays the audio response while simultaneously highlighting the step on the visual display.

4.3.3 Functional Requirements

- REQ-9: The system shall stream user audio input to the external voice processing service in real-time.
- REQ-10: The system needs to support navigation commands including "Next," "Previous," "Repeat," and "Start Over."
- REQ-11: The system must synchronize the visual state of the application with the audio response, ensuring the current spoken step is visually highlighted.
- REQ-12: The system needs to handle ambiguous or unrecognizable voice inputs by requesting clarification from the user via a synthesized voice prompt.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

For the real time system (RTS), BYF will be up 99.9% of the time while the other 0.01% of the time will be dedicated to making changes and updated to BYF.

5.2 Safety Requirements

Admins should not share their credentials with any malicious actors to prevent any harm that can be done to the web application.

5.2.1 User privileges

To deter malicious users, the current software development shall have users passively protected with a simple implementation of user classes within the system. The two classes that are primarily used are User and Admin. Mostly, if not all, unknown users are automatically put in the User class unless authorized to be an admin, which has more privileges in the system. Users have a more surface-level interaction with the system.

5.2.2 Encryption

The system shall ensure the confidentiality of data if read by a malicious user, preventing attacks.

5.2.3 Tokens

The system shall ensure the safety of users by providing tokens to users to prevent unauthorized guests. Tokens shall prevent data breaches or system compromises.

5.3 Security Requirements

5.3.1 Password-Hashing

The system shall have security measures that are implemented into the system to protect users such as password-hashing as well as salting. A module that adds a random string of characters before the initial password is created. Having both should protect passwords if data is compromised, and also distinguish differences between users who have the same passwords.

5.3.2 SQL injection prevention

The system shall prevent data from being unwillingly acquired, countermeasures are implemented into the system, such as parameterized query insertion rather than basic SQL queries. This is a more defensive way of coding while creating a database to help prevent injections from occurring.

5.3.3 Authentication

The system shall require users to authenticate using a valid email and password before accessing the protected features such as saved recipes and cooking sessions.

5.3.4 Voice Data Privacy

The system shall request user permission before accessing the microphone. Voice input shall be used only for real-time command processing and shall not be permanently stored unless explicitly required by the user.

5.3.5 Security Logging

The system shall log security-related events such as failed login attempts, password changes, and administrative actions. Logs shall not contain plaintext passwords or sensitive personal information.

5.3.6 Authorization and Access Control

The system shall implement role-based access control. Creating, editing, and deleting shared recipes shall be restricted to authorized admin users only. Regular users shall only be permitted to modify their own personal data.

5.4 Software Quality Attributes

5.4.1 User Quality of Life

The system shall provide clear and concise instructions to users while operating on recipes at users' request. It shall provide users with essential information while cooking.

5.4.2 Developer Quality of Life

The system shall allow developers to use the Gemini Voice API for generating text-to-speech more effectively, rather than creating a text-to-speech system.

5.5 Business Rules

Chef accounts will have functional access to recipes posted by a head chef such as:

- selecting a recipe of the user's choice
- playing a hands-free-voice application to read out recipes
- rating a recipe up to five stars.
- commenting on a recipe
- changing email or password.

Head chef accounts have the same privileges as chef accounts except also:

- banning users for inappropriate behavior.
- creating recipes for other users and admins.
- viewing statistics of recipes.

Appendix A: Glossary

Admin	*An individual who manages the system.*
Agent	*An autonomous task, that acts for the user or another program.*
Analysis Model	*A link between the system description and the design model.*
Application	*A software that helps users perform tasks on a system.*

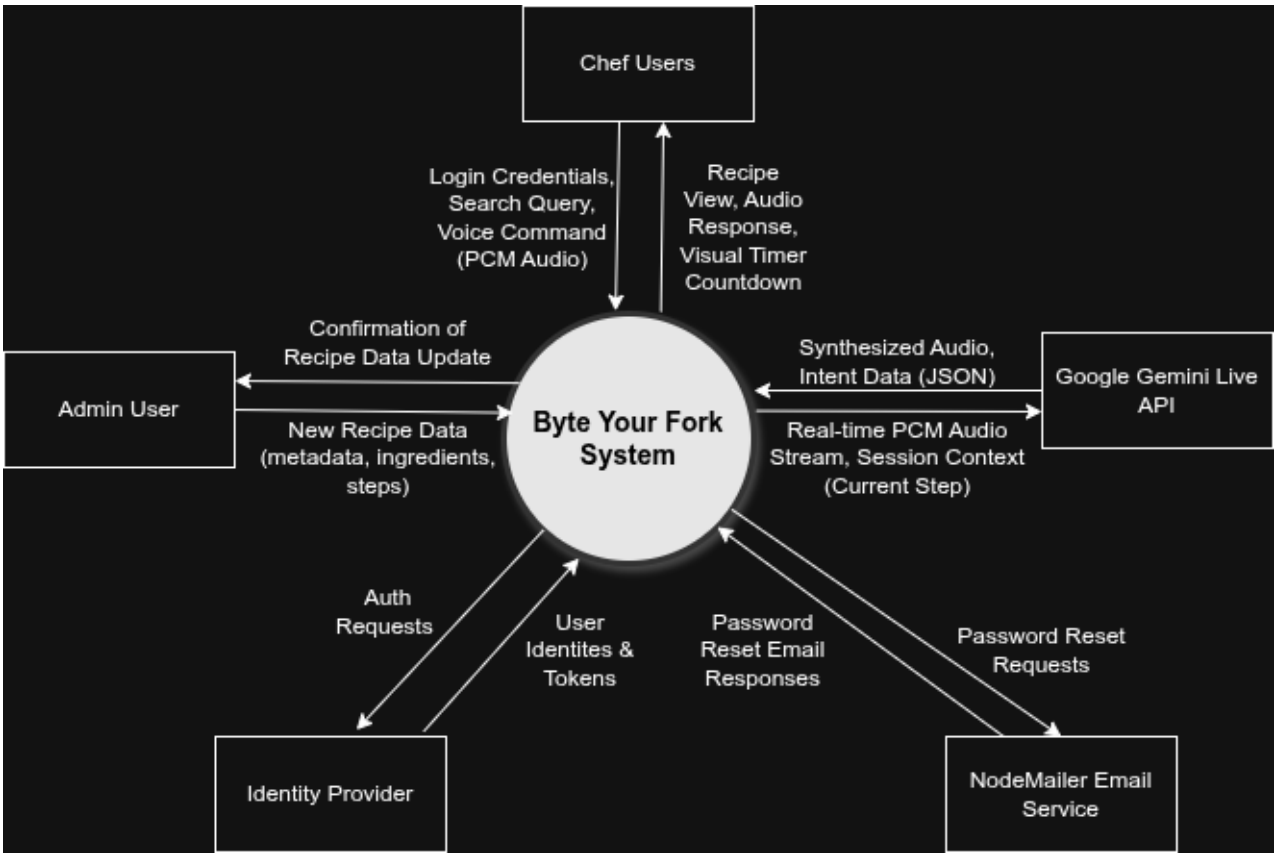
Application Programming Interface (API)	*A software that helps users perform tasks on a system software.*
Backend	*Operations and functionalities that work hidden behind the program.*
Broadband	*A high-speed internet connection that allows for the rapid transfer of large data.*
Client	*A high-speed internet connection that allows for the rapid transfer of large data.*
Database	*An organized collection of structured data.*
Encrypt	*Process into converting data into secure code.*
External Integration Point	*Automates internal software connections.*
Federated	*Multiple different units work together as one single entity.*
Filter Chips	*A word tag that narrows selection.*
Framework	*A guide for the software being built.*
Hashing	*Generating a key to an index with a defined function.*
HyperText Markup Language (HTML)	*The standard language used for web applications.*
HyperText Transfer Protocol Secure (HTTPS)	*A protocol that defines how data is transferred across the internet.*
Interface	*A set of rules that allows two separate components to interact.*
Interoperability	*The ability for two systems or software's to exchange and use data.*
JavaScript Object Notation (JSON)	*Lightweight format for storing and transporting data.*
Latency	*The time delay between user actions and the response of a device.*
Metadata	*Data that is describing data.*
Multipurpose Internet Mail Extension (MIME)	*Extends the format of email messages allowing them to include more than just text.*
Optimize	*Fine-tuning software to maximize speed, efficiency and performance.*
Overhead	*The cost of system resources.*
Pulse Code Modulation (PCM)	*Method to represent analog signals.*

Querying Mechanism	*Technical processes to work on the data from databases.*
Relational Database Management System (RDBMS)	*Software used to control databases.*
Sample Rate	*Times per second an analog audio signal is measured.*
Scope	*The visibility of objects or data with the program.*
Serialized	*Converting complex data that can be easily stored by the computer.*
Server	*Provides data and services to clients.*
Speech Synthesis Capability	*To convert human text to audible output.*
Simple Mail Transmission Protocol (SMTP)	*A protocol that controls how emails are sent over the internet.*
Transactional Data	*Data that is information for transactions.*
User	*The customer.*
User Context	*The Individual using the application circumstances of the user.*
User Data	*Information about the user and their interactions with the software.*
User Interface (UI)	*Visual interactive parts for the user to interact with the software.*
User-facing	*Parts of the product that interact directly with the user.*
Viewports	*The visible window that the user can see through.*
Web Interface	*The visual layers of websites that allow user interactions.*
Web-based communication	*The exchange of messages through the internet.*
Web Socket	*A protocol that allows real-time communication between two clients.*

Appendix B: Analysis Models

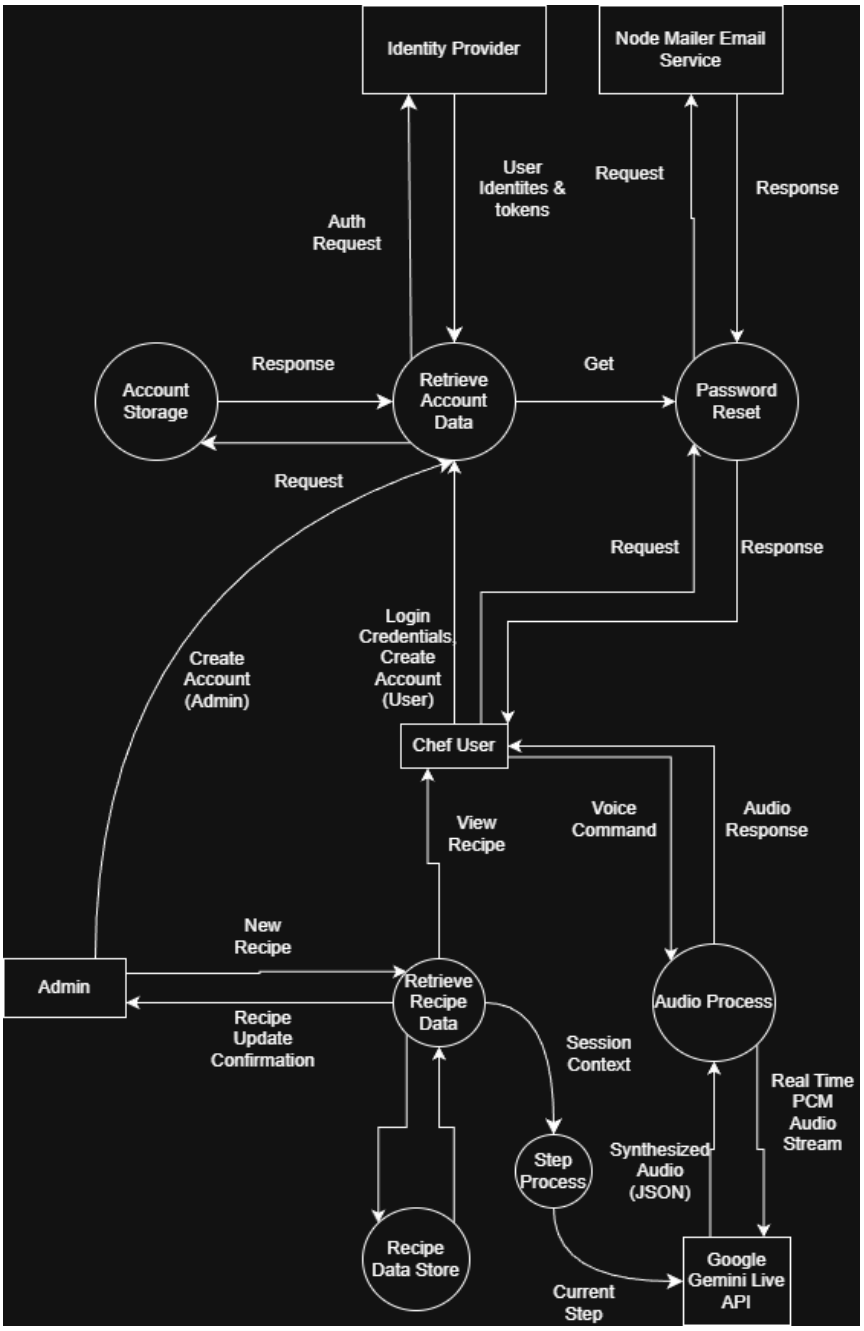
System Context Diagram

The context diagram below serves as a high-level architectural blueprint that defines the boundary between the Byte Your Fork black box system and the external entities around it. It shows the inputs and outputs from Byte Your Fork to external entities.



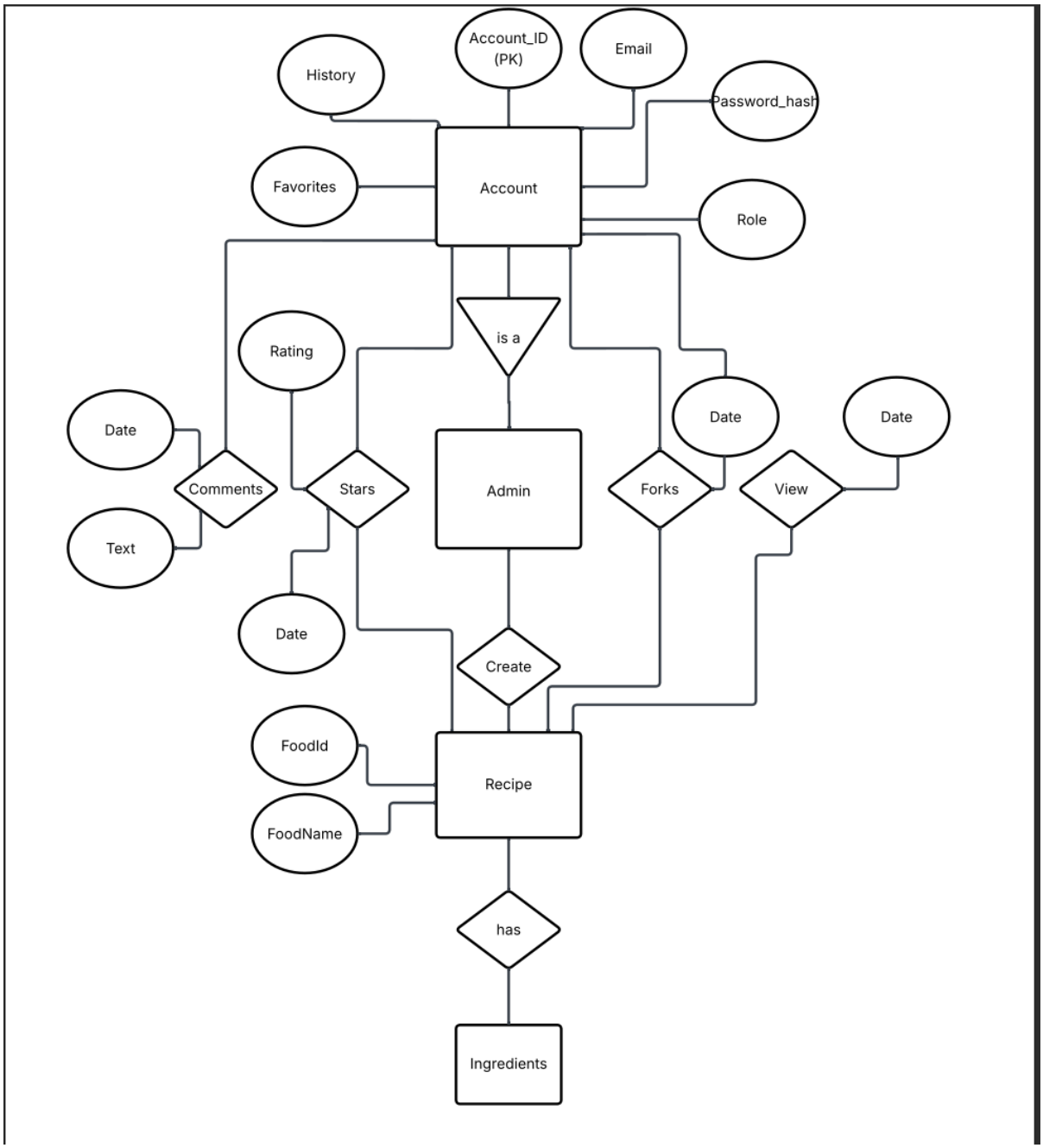
Data Flow Diagram

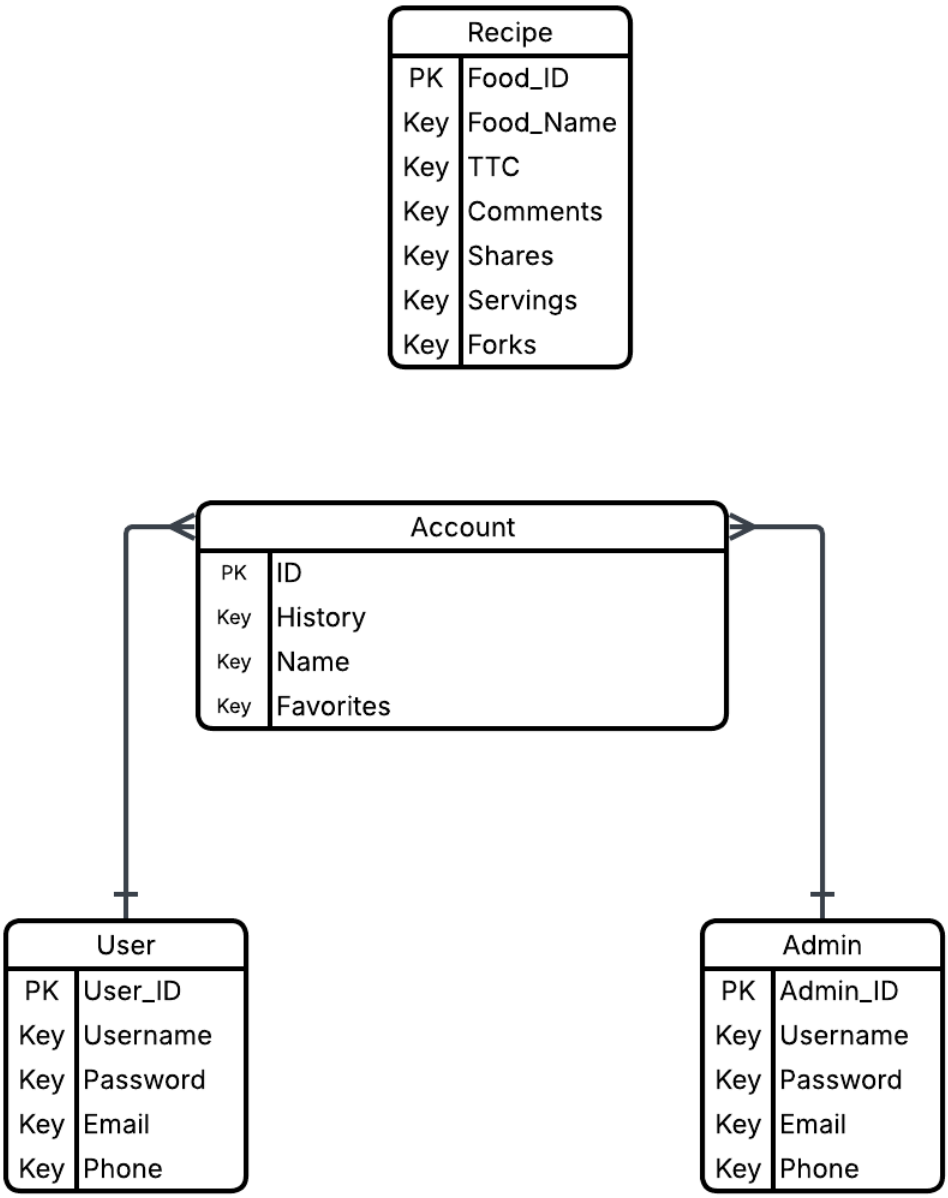
The Level 1 diagram provides a more granular view by decomposing the system into its primary functional processes, illustrating how data flows between internal modules and persistent data stores.

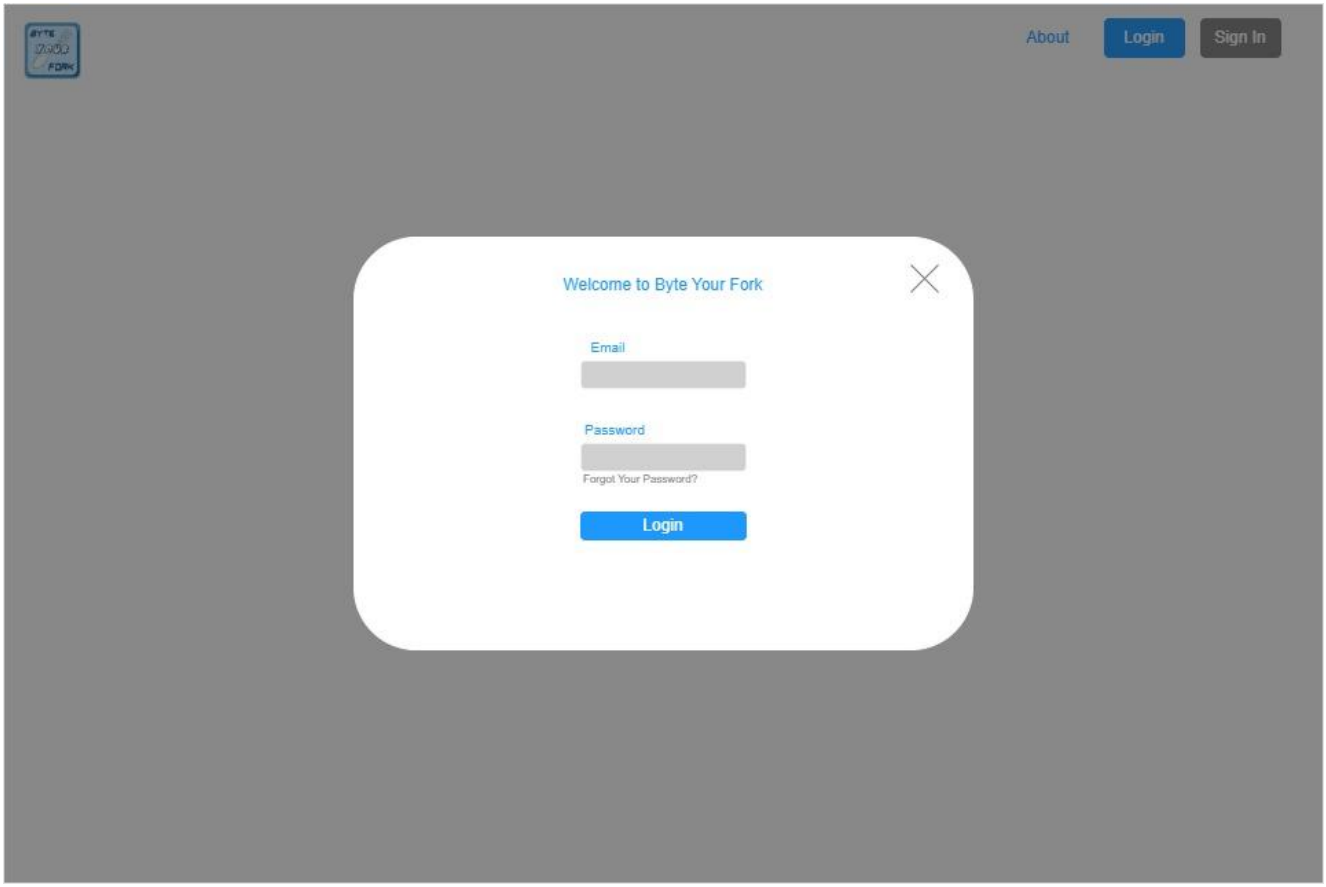


ERD Diagram

The Entity Relationship Diagram illustrates the normalized, BNF compliant database architecture for the system.




Recipe
PK Food_ID
Key Food_Name
Key TTC
Key Comments
Key Shares
Key Servings
Key Forks
Account
PK ID
Key History
Key Name
Key Favorites
User
PK User_ID
Key Username
Key Password
Key Email
Key Phone
Admin
PK Admin_ID
Key Username
Key Password
Key Email
Key Phone

Sample Login Page

The sample authentication screen mockup demonstrates the primary user access point for the application.